Mock Exam SS/2024

08.07.2024

Name:	**Nord**	First Name:	**Frank**
Study:	**Informatik**		
Matr.No.:	**123456**	Exam Number:	**23**

Information: (Please read carefully!)

- Sign the declaration below. Without your signature, we retain the right to not grade your solutions.
- The exam comprises **6 tasks on 16 pages**, plus **2** additional pages for notes.
- Duration of exam: **120 minutes**.
- The exam consists of **64** points. **32** points are required to pass.
- Answer the questions in the corresponding solution boxes. If you need more space, use a new sheet of paper. You can use the additional pages at the end of the exam. If needed, you can get additional paper from the supervisor. Write down your name as well as your matriculation number on each additional paper, also make sure to note to which task your solution belongs. Put a reference to where the solution is to be found at the original task.
- Questions should be answered in English. If you don't know how to answer a question in English, German is allowed.
- At the end of the exam, the examination sheets together with additionally used paper has to be returned.
- **No additional aids** (notes, calculators, cellphones, smartwatches, ...) are allowed.
- Use a pen for writing down your solutions, **pencils are not allowed**. Also, do not write with red or green ink.
- Place an ID card on the table.

I hereby declare that I have read and understood the information above, that the provided answers are solely results of my own work and that I am feeling healthy enough to participate in the exam.

Signature

Summary:

Task	1	2	3	4	5	6	Σ	Grade
Points	9	13	12	11	9	10	64	
Points achieved								

This page is intentionally left blank.

Solution 1 (General Concepts) **(1 + 2 + 3 + 3) = 9 Points**

a) (1 Point) What are the two protection domains (or modes) used by most operating systems architectures?

user and supervisor

b) (2 Points) What is the difference between these two modes?

user is forbidden from executing certain instructions, e.g., interacting with devices, and accessing some supervisor-only memory regions.

c) (3 Points) Name two operating system architectures we discussed in the lecture, and detail which components run in which protection mode.

Monolithic: user apps in *user*, everything else in *supervisor*.
Microkernel: minimal components in supervisor (basic IPC, virtual memory, basic scheduling), everything else in user (drivers, fs, etc.)
Hybrid kernel: mix of monolithic and microkernel. Monolithic, with some components moved to user for safety.
Unikernel: everything in supervisor mode.

d) (3 Points) What are the two types of hypervisors? Explain the difference between them. You can draw a diagram to help your explanation.

Type-1: bare metal. Hypervisor executes directly on the hardware (it is an OS), in a privileged domain, e.g., ring -1. OSes run on top of it, "normally", with the kernel running in supervisor mode (ring 0) and the rest in user mode (ring 3).

Type-2: hosted. The hypervisor is a normal application running on top of an operating systems, e.g., Linux. Potentially with the help of some kernel components (e.g., KVM). VMs run as normal applications (ring 3), side by side with "regular" host processes.

Solution 2 (Program Execution) **(2 + 2 + 1 + 2 + 6) = 13 Points**

a) (2 Points) When a process is started, the operating system creates an execution environment for it, i.e., its memory layout. Variables are stored in two different areas - firstly in the `Stack` and secondly in the `Data` section. *For both structures, specify a type of variable that is stored there.*

Stack:

local variable, context (stack frame), environment variables

Data Section:

static local variables, global (initialized) variables, constant strings

In addition to the `Data` section, the `BSS` section and the `HEAP` exist. *For both structures, specify a type of variable that is stored there.*

BSS:

static uninitialized variables

Heap:

dynamically allocated memory (malloc)

b) (2 Points) In the lecture, you learned about two levels of threads. *Specify these two levels.*

Thread level 1:

User-Thread

Thread level 2:

Kernel-Thread

How can threads be mapped from one type to the other? *Name one* of the three possible variants and *briefly describe* the meaning of this mapping.

Many-to-One: Several user threads are mapped to one kernel thread.
One-to-One: Each user thread is mapped to its own kernel thread.
Many-to-Many: The user threads are mapped to a pool of kernel threads.

We execute the following C program:

```
#define N 5

int main(int argc, char **argv) {
        int i;

        for (i = 0; i < N; i++) {
                int ret = fork();

                if (ret < 0) {
                        printf("Error\n");
                        return 255;
                }
                if (ret > 0) {
                        break;
                }
        }

        return 0;
}
```

c) (1 Point) How many processes are created during the execution of this program?

6 processes (main + the five calls to fork() in the loop)

d) (2 Points) Draw the tree of processes created by this program.

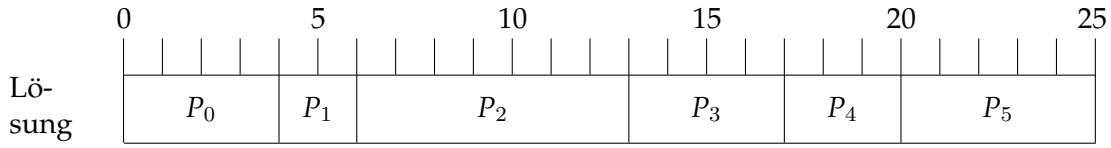
e) (6 Points) Given a computer system with one CPU and six processes $P_0, \ldots, P_5$. In the table below, the arrival time and the execution time (time required by the CPU to execute the program) are given for these processes.

An arrival time of t means that the process is already in the `ready` state, waiting for CPU allocation, and can be immediately considered by the scheduler, i.e., it can use the CPU in the same time unit. The context switch between two processes takes no time. If a process occupies the CPU for the specified execution time, it immediately leaves the system without using any more CPU time.

Illustrate the CPU allocation based on the provided chart for each scheduling strategy. The numbers above each chart indicate time points and are provided only to facilitate the creation of the solution. You must also calculate the average waiting time for each strategy for your schedule. Perform your calculations in the field labeled "Calculation" and enter the final result in the field labeled "Result".

Process	Arrival Time	Execution Time
P_0	0	4
P_1	1	2
P_2	3	7
P_3	4	4
P_4	6	3
P_5	7	5

Strategy: **First-Come First-Served (FCFS)**



Average Waiting Time:

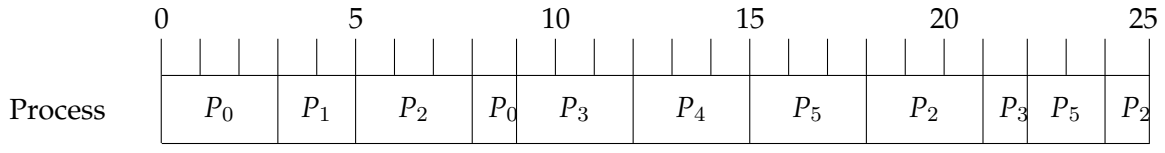
Calculation:

Waiting time of process P_i is the duration between the arrival time of P_i and the time P_i is scheduled. We sum all waiting times, and divide by the number of processes to get the average waiting time.
$\frac{0+3+3+9+11+13}{6} = \frac{39}{6} = 6.5$

Result:

6.5

Strategy: **Round-Robin** *Quantum = 3*



Average Waiting Time:

Calculation:

Two potential definitions of waiting time here, both accepted:

- Duration between arrival time and first time of schedule: $\frac{0+2+2+5+6+8}{6} = \frac{23}{6} = 3.8333$
- Total duration spent waiting, adding all the waiting times due to context switches to the previous sum: $\frac{5+2+15+14+6+12}{6} = \frac{54}{6} = 9$

Result:

3.8333 or 9

Solution 3 (Memory Management) **(2 + 1 + 4 + 1 + 1 + 1 + 2) = 12 Points**

a) (2 Points) *Explain the differences between the following pairs of terms:*

Internal and external fragmentation:

Internal fragmentation refers to memory areas that are assigned to a process but are not used by it, e.g., unused memory inside a page.
External fragmentation refers to memory areas that are not assigned to a process but are too small to use, e.g., unused memory between segments.

Logical and physical address:

Physical addresses indicate where a page/segment can be found in the main physical memory, while logical/virtual addresses refer to the address space of an individual process.

b) (1 Point) Which two parts does a *logical address* consist of when using segmentation?

segment number & Offset / Address within Segment

c) (4 Points) In a system using segmentation to manage memory, the following segment table is given for a process:

Segment	Base	Limit
0	5300	400
1	1200	300
2	300	200
3	1500	400
4	3300	200

The following logical addresses are now requested by the process. *Specify what the MMU would return in each case.*

(0, 453):	segmentation fault	(1, 0):	1200
(3, 400):	segmentation fault	(4, 33):	3333

Which *logical addresses* would each point to the following physical addresses?

5692:	(0, 392)	1500:	(3, 0)

d) (1 Point) In a system using paging to manage memory, with the following specification:

- Address size: 32 bits
- Page size: 2 kiB
- Page table entry size: 4 B

What is the size of address spaces in GiB? How many pages does an address space contain?

Addresses are 32 bit long, so 2^{32} bytes are addressable. Thus, the address space is $2^{32}B = 4GiB$. A page is $2kiB = 2^{11}B$, which means that the number of pages needed to cover the whole address space is: $nr\_pages = \frac{2^{32}}{2^{11}} = 2^{21} pages$

e) (1 Point) How many page table entries are needed?

We need as many page table entries as pages, so 2^{21} page table entries.

f) (1 Point) How many page table entries fit in a single page?

A page table entry is $4B = 2^2B$ long, a page fits $2KiB = 2^{11}B$ of data.
Thus, a page fits: $\frac{2^{11}}{2^2} = 2^9 = 512$ pages.

g) (2 Points) Processes work with *logical* (*virtual*) addresses, which are mapped via the Memory Management Unit (MMU) to *physical* addresses of the main memory. *Judge whether the following two statements are true by checking one of the two boxes AND justify your answer in the text box provided.*

Statement 1: The same logical address of two different processes can be mapped to different physical addresses.

	yes	no
The claim is true:	■	☐

Justification:

True. Processes usually require exclusive memory pages (stack, heap). However, as they all use the same logical address space, the logical addresses are inevitably mapped to disjoint physical addresses. Address spaces are independent from each other.

Statement 2: Different logical addresses of two different processes can point to the same physical address at the same time.

	yes	no
The claim is true:	■	☐

Justification:

True, e.g., with shared memory. In this case, both processes must access the same physical memory; however, the processes / the system decide at which point this shared memory segment is integrated in the logical address space.

Solution 4 (File Management) **(3 + 2 + 1 + 1 + 4) = 11 Points**

a) (3 Points) Given a fictional 32-bit file system based on inodes. The block size is 40 bytes. An inode contains 5 direct pointers, one single indirect pointer, and one double indirect pointer.

What is the maximum file size in this file system? Provide your calculation!

32-bit pointer, which is 4 bytes, so 10 pointers per block.

$$5 \cdot 40 + 10 \cdot 40 + 10^2 \cdot 40 = 200 + 400 + 4000 = 4600 \text{ bytes}$$

or

$$(5 + 10 + 10^2) \cdot 40 = 115 \cdot 40 = 4600 \text{ bytes}$$

How many blocks are required for a file with a file size of 240 bytes? *Justify* your answer!

$\frac{240}{40} = 6$ blocks are required for the file content. However, there are only 5 direct pointers, so we need an additional indirect block. In total, 7 blocks are required.

b) (2 Points) *Name four entries* of different types that can be found in an inode.

- Owner (uid) and group (gid)
- Access permissions (user, group, other)
- Access time (atime), Modification time (mtime), Change time (ctime)
- File size
- Hardlink count
- Inode type
- Data pointers
- Pointers (direct, single indirect, double indirect, triple indirect)

c) (1 Point) What does a file descriptor represent?

It describes the state of a currently open file.

d) (1 Point) Name the three information a file descriptor should at least contain.

Access mode, position and file identifier, e.g., inode

Consider the following program:

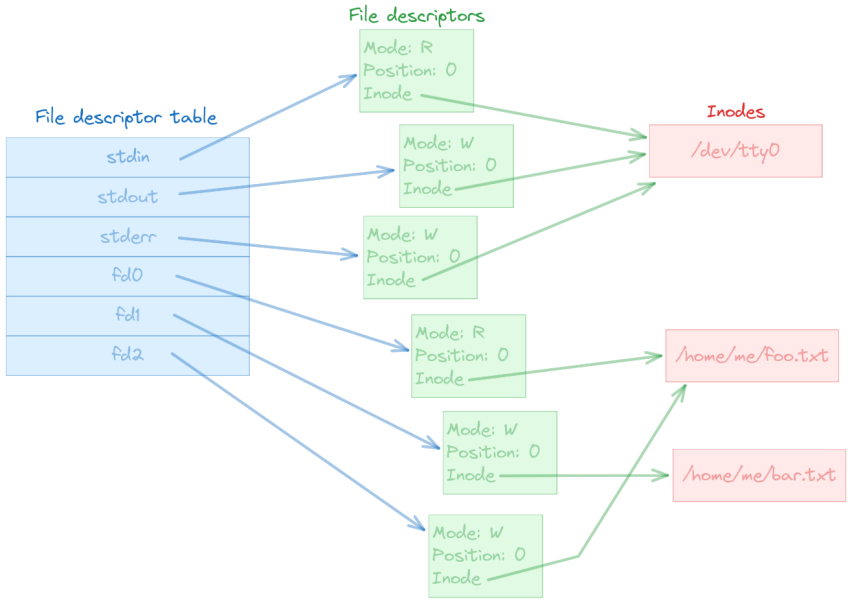
```
int main(void) {
        int fd0, fd1, fd2;

        fd0 = open("/home/me/foo.txt", O_RDONLY);
        fd1 = open("/home/me/bar.txt", O_WRONLY);
        fd2 = open("/home/me/foo.txt", O_WRONLY);

        /* do things */
}
```

e) (4 Points) Draw the structures stored by the kernel for the process running this program after the last `open`. You only need to represent structures related to file management in the process control block, as well as structures in the file system. You can assume that the file system hosting the files uses inodes.

The PCB contains the file descriptor table of the process, with 6 entries: stdin, stdout, stderr, fd0, fd1, fd2, each pointing to a struct file descriptor. These file descriptor structures contain the access mode, position and a file identifier, with fd0 and fd2 pointing to the same inode, and the standard in/out/err pointing to the terminal device file.



Solution 5 (Input-Output Devices and Interrupts) **(1 + 1 + 2 + 5) = 9 Points**

a) (1 Point) How do drivers *simplify* kernel programming?

Drivers offer a standardized interface for devices of one type and translate these into device-specific commands. (The kernel only sees "Printer with function A, B, G, K" and not "Pixel-Print ProximaXpress X3000 Series II Deluxe Edition").

b) (1 Point) *Explain briefly* why Direct Memory Access (DMA) is used.

DMA allows the CPU to not be blocked during data transfers between I/O devices and the main memory, allowing it to perform other tasks during that time.

c) (2 Points) *Describe briefly* the difference between Port-mapped and Memory-mapped I/O.

In Port-mapped I/O, the registers of a device are identified by a port number.
They are accessed with io-specific instructions.

In Memory-mapped I/O, a portion of the physical address space is reserved for I/O devices. Each register of a device is assigned a specific address range, which is no longer available for main memory. They are accessed through regular instructions operating on memory.

d) (5 Points) Assume a hard disk has **2000** cylinders, numbered from **0 to 1999**. The disk head (read/write head) is currently serving an I/O request on cylinder **288**, and the previous request was served on cylinder **312**. Several other requests are waiting to be served; the cylinders to be accessed in the following are sorted according to the order in which the requests were made to the operating system: **350, 860, 210, 1633, 1885, 1100, 930, 1441**.

Based on the current position, calculate the resulting schedule and the total distance (in cylinders) that the disk head has moved for the scheduling strategies Shortest Seek First (SSF) and the elevator C-SCAN.

Enter the results in the respective solution boxes! Detail your calculations in the "Calculations" boxes.

SSF:

Schedule: [288, 350, 210, 860, 930, 1100, 1441, 1633, 1885]

Total seek time: 1877

Calculations:

We find the closest cylinder from the current position.
Schedule: [288, 350, 210, 860, 930, 1100, 1441, 1633, 1885]

Total seek time: 1877

C-SCAN:

Schedule: [288, 350, 860, 930, 1100, 1441, 1633, 1885, 210]

Total seek time: 3921

Calculations:

The head served 312 before, went all the way to the last cylinder, then back to cylinder, then to the current request 288. It then serves all requests until the last cylinder 1999. Then, wee circle back to cylinder 0, and serve all requests again from 0 to 1999, here, only request 210 remains.

C-SCAN-Schedule: [288, 350, 860, 930, 1100, 1441, 1633, 1885, (keep going until 1999, then back to 0), 210]

Total seek time: 3921

Solution 6 (Concurrency & Inter-Process Communication) (1 + 2 + 4 + 1 + 2) = 10 Points

a) (1 Point) What is a critical section?

A critical section is a portion of code that should be accessed by at most one thread at a time to avoid race conditions.

b) (2 Points) Define what a semaphore is. List the type of information it contains, the primitives it provides and what each primitive does.

An atomic integer K >= 0
wait(): block on the semaphore until K > 0, then decrement K atomically
signal(): increment K atomically

c) (4 Points) You have two processes communicating through a shared memory buffer (an array) that can contain 10 elements. The first thread writes into the buffer, while the second reads from it. What synchronization mechanism(s) should you use to make sure everything works fine?

Two integers: reader and writer indexes, that loop from 0 to 9, both initialized to 0;
two semaphores: number of available elements (initialized to 0) sem_avail, number of free spaces (initialized to 10) sem_free.
When the writer wants to write, it waits on sem_free, then writes the element at index writer_idx, increments writer_idx, then post sem_avail.
When the reader wants to read, it waits on sem_avail, then reads the element at index reader_idx, increments reader_idx, then post sem_free.
Increments to indexes are modulo 10.

Assuming the following code, where `worker()` and `dispatcher()` can be executed in parallel:

```
mutex_t res1, res2;

int worker(int id) {
        lock(res1);
        lock(res2);
        do_something();
        unlock(res2);
        unlock(res1);
}

int dispatcher(void) {
        lock(res2);
        lock(res1);
        dispatch();
        unlock(res2);
        unlock(res1);
}
```

d) (1 Point) Why can this program cause a deadlock?

Locks are taken in a different order by workers and dispatchers.

e) (2 Points) Which condition for a deadlock can be easily removed to solve the problem?

Circular wait condition by having dispatchers lock res1 then res2.